Alfred

A software butler that points at the next thing to click

Interview study guide: what was built, how it fits together, and why each decision was made

Written in plain language. No coding knowledge needed.

Project timeline: 14 Aug 2026 to 8 Oct 2026 · Built by AdibOmi

Start here

This guide is ordered so you can stop at any point and still have something useful to say. If you only have ten minutes, read this page, the interview questions (section 10) and the cheat sheet on the last page.

Section	What you get from it
1. The 30-second pitch	Words you can say almost exactly as written
2. The problem	Why the project exists and who it is for
3. What Alfred does	The three features, with screenshots
4. The journey	How the idea changed over five versions, and why
5. How it is built	The architecture, explained with a restaurant analogy
6. One question, start to finish	What happens behind the scenes when someone asks for help
7. Key decisions	Each big choice, the options, and the reason
8. Speed, privacy, quality	How it was made fast, safe and tested
9. Limits and next steps	What it does not do yet, and what you would add
10. Interview questions	Likely questions with sample answers
11. Glossary & cheat sheet	Every technical word in plain English, and a one-page summary

1. The 30-second pitch

> **Memorise this**
>
> *"Alfred is a small desktop assistant for people who get stuck in software they don't know, like a student opening Excel for the first time. You press a shortcut and type what you're trying to do, for example 'how do I make a pie chart?'. Alfred takes one screenshot, works out where you are, and a ghost cursor glides to the exact button you need to click, with a short instruction beside it. You click, press 'next step', and it guides you one step at a time until you're done. It also keeps your to-dos and reminders, and emails you a weekly cheat sheet of everything it taught you."*

Then add one sentence about how it's built:

> *"It's an Electron and React desktop app talking to a Python FastAPI server, which uses free AI models from Groq or Google Gemini. I built in accounts, a database, caching, scheduled jobs, PDF email reports and AI tool-calling, and I spent a lot of effort making it fast, about one second from question to pointer, and private: no screenshots are ever saved."*

2. The problem

Students and people new to office work get stuck in unfamiliar software: Excel, Word, PowerPoint, university portals. Online tutorials say things like **"go to Insert, then Chart"**, but the person can't find *Insert* on their own screen. Their version looks different, a menu is hidden, or there are just too many buttons. So they give up, or wait for someone to help.

The target user: a student who opens Excel for an assignment and doesn't know where anything is.

The key insight: the hard part isn't knowing *what* to do, it's finding *where* it is. A text answer still leaves you searching. Alfred **shows** you, by pointing at the actual button on your actual screen.

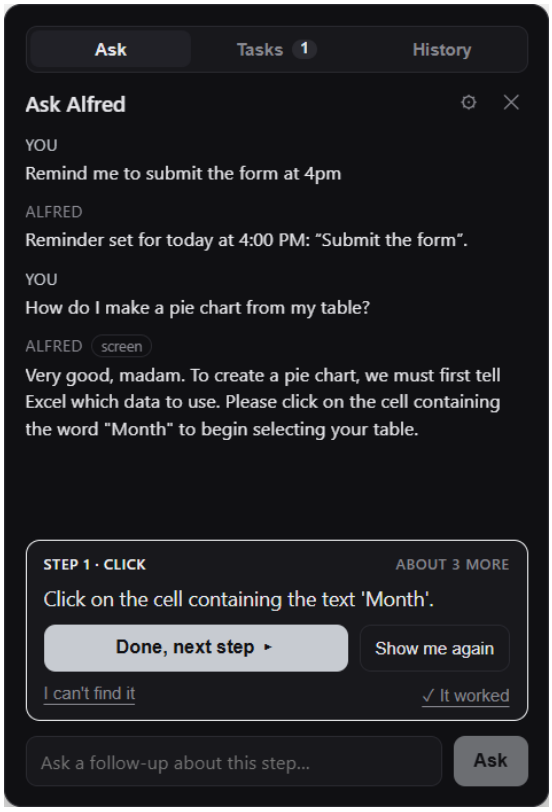
> **Say it in the interview**
> "Tutorials tell you what to click, but not where it is on your screen. Alfred closes that gap by pointing at it."

3. What Alfred does

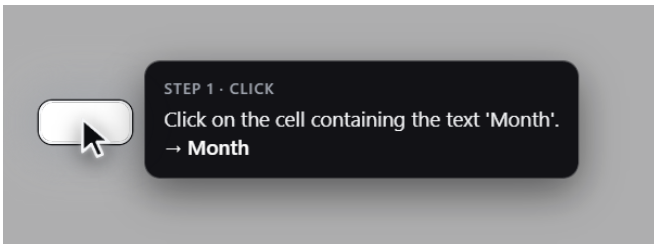
Alfred lives as a small floating icon (a bat-wing bow tie, a nod to Batman's butler) in the corner of the screen. Press **Ctrl + Shift + A** or click it, and a panel opens with three tabs.

Feature	What it does for the user
Ask (the main feature)	You describe what you want to do. Alfred looks at your screen and a ghost cursor points at the next button, with a one-line instruction. Press **Done, next step** and it looks again and points at the next one. **I can't find it** explains the same step differently. It never clicks for you.
Tasks	To-dos and reminders. Add them by hand, or just type "remind me to submit the form at 4pm". You get a pop-up notification when it's due, or an email if Alfred wasn't running.
History & reports	Every task Alfred walked you through is saved with its steps. Each Monday you get a PDF "cheat sheet" of your week by email, so next time you can do it alone.

What it looks like



The panel: a reminder set by typing, then a guided step in Excel



The ghost cursor and ring on the real screen, with the instruction beside it

Notice the personality: Alfred talks like a polite butler ("Very good, madam…"). Notice also the **"screen"** label on the reply. It tells the user that this answer used a screenshot, while the reminder above it did not. That's part of the privacy design (section 8).

Why it never clicks for you

This was a deliberate choice, not a missing feature. If Alfred pointed and you click, two good things happen: you **learn** the software (the whole point), and Alfred **can't click the wrong thing** and cause damage, such as deleting a file or submitting a form early.

> **Say it in the interview**
>
> "Alfred points, you click. That's how you learn, and it means a mistake by the AI can never do harm."

4. The journey: how the idea changed

Interviewers love this part because it shows judgment. The project went through five versions in about two months. Each change happened for a reason you can explain.

Version	What it was	Why it changed
1. "Batcomputer" dashboard 14 Aug	A big personal dashboard: reminders on a timeline, file search, a Claude AI chat, and progress cards pulling from GitHub, LeetCode and a gym log.	It tried to do everything, so nothing was special. It also collected a lot of personal data.
2. The pivot 26 Aug	Threw almost all of it away (5,500 lines deleted). Alfred became one tiny floating icon: ask a question about your screen, get numbered text steps back. Strict privacy: no database, no history, screenshots never saved.	Focus. One clear job, done well, with privacy as a selling point.
3. Smart assistant 23 Sep	Brought back to-dos and reminders, but behind the same text box. The AI now chooses between "look at the screen" and "manage a task" using *tools* (explained in section 7).	Users want one place to type. Letting the AI choose also meant no screenshot is taken for a to-do.
4. Guided pointer 7 Oct	The big upgrade. Added a real server, accounts, history, weekly reports, and the ghost cursor that *points* instead of describing. Moved from a paid AI (Claude) to free ones (Gemini, Groq).	Text steps still left users hunting for buttons. A server was needed for accounts, history and emails, and free models mean anyone can run it.
5. Speed & accuracy 8 Oct	Made it fast (about 1 second) and accurate: faster Windows screenshots, on-screen text reading (OCR) to place the pointer exactly, skipping unneeded AI calls, and the app starting its own server automatically.	A guide that takes 5 seconds per step feels broken. Fast models placed the pointer loosely, so OCR fixes the aim.

Say it in the interview

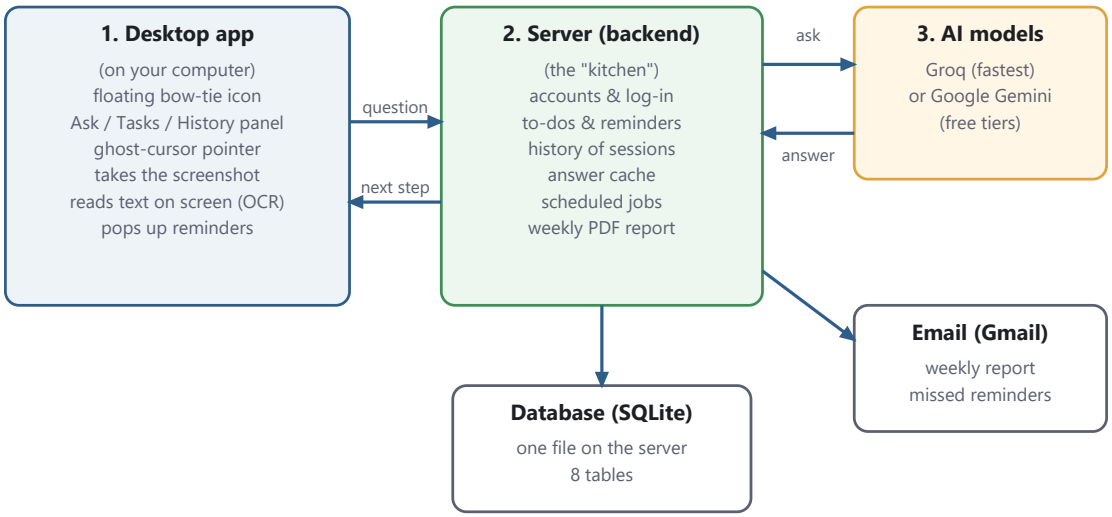
"My first version tried to be everything, a whole dashboard. I realised the one genuinely useful idea was helping people find things on their screen, so I cut 5,000 lines and rebuilt around that. Then I noticed that even good text instructions left people searching, so I made Alfred point instead."

The project was also built as a **capstone** for a program (FlyRank) that asked for at least five of seven backend concepts. Alfred uses all seven (see section 7, "The seven concepts"). The one-page plan in *docs/PLAN.md* set milestones: plan, backend core, AI on the server, desktop, then reports and jobs.

5. How it is built

Think of Alfred as a **restaurant**:

- The **desktop app** is the dining room and waiter. It is what you see and touch: the icon, the panel, the pointer. It takes your order (your question) and brings back the food (the next step).
- The **server** (also called the backend) is the kitchen. You never see it, but it does the real work: checks who you are, remembers your tasks and history, and asks the AI for help.
- The **AI models** are a specialist chef the kitchen calls on. Alfred doesn't own them; it sends them a request over the internet and gets an answer back.
- The **database** is the pantry and the order book: everything that must be remembered.



The three main parts of Alfred and how they talk to each other

The technology, in one table

Part	Tools used	Plain-English meaning
Desktop app	Electron, React, TypeScript, Vite	Electron lets you build a desktop app with web technology (the same tech as websites). React builds the screens. TypeScript is JavaScript with safety checks.
Server & database	Python, FastAPI, SQLite	FastAPI is a popular, fast way to build a web server in Python. SQLite is a database that is just one file, with nothing extra to install.
AI	Groq or Google Gemini, Windows OCR	Free AI services: Groq is the fastest, Gemini the most precise. OCR (reading text from an image) is built into Windows and places the pointer.
Jobs & reports	APScheduler, fpdf2, Gmail	APScheduler runs tasks on a timer, like an alarm clock; fpdf2 creates PDFs; Gmail sends them.

Say it in the interview

"It's a client–server design: an Electron app, a Python FastAPI server with SQLite, and free AI models called from the server, so the API key never sits on the user's machine."

6. One question, start to finish

Here is what happens when a user types **"How do I make a pie chart from this table?"** and presses Enter.

#	What happens	Why it's done this way
1	The app **immediately** starts taking a screenshot, in parallel with everything else.	Saves time. If the screenshot turns out not to be needed, it's thrown away without being sent.
2	Alfred checks the wording. "How do I…" can only be about the screen, so it skips asking the AI what the message is about.	Removes one whole AI call (one round trip) for the most common kind of question.
3	The screenshot is shrunk and sent to the server with the goal. Alfred's own windows are left out of the picture.	Smaller images are faster. Hiding Alfred means the AI sees your app, not Alfred.
4	The server checks its **cache**: "Have I answered this exact question on this exact screen before?"	If yes, the answer comes back instantly, without asking the AI.
5	If not, the AI vision model is asked for **exactly one** next step and the name of the thing to click (e.g. "Insert").	One step at a time, on a fresh screenshot, keeps Alfred correct when menus open or dialogs pop up.
6	Meanwhile, on the user's computer, Windows OCR reads all the text on the screenshot (about 60 ms).	Happens while waiting for the AI, so it costs no extra time.
7	The AI says "click Insert"; OCR knows exactly where the word "Insert" is. Alfred **snaps** the pointer onto it.	Pixel-accurate. If the text can't be found, Alfred shows the step in words rather than pointing at the wrong thing.
8	A ghost cursor glides to the button and draws a ring, with the instruction beside it.	Movement draws the eye. On Windows it can also glide your real mouse pointer there.
9	The user clicks and presses **Done, next step**. A fresh screenshot is taken and the loop repeats.	Each step is saved, building the history and the weekly cheat sheet.

And if the user types "remind me to submit the form at 4pm"?

The server gives the message to a chat AI that can use **tools**: create, update, complete or delete a task, or "look at the screen". The AI picks *create task*, the server saves it, and Alfred replies "Reminder set for today at 4:00 PM". **No screenshot is ever sent**, because the AI never asked to look.

7. Key decisions and why

This is the most important section for an interview. For each decision: what was chosen, what else was possible, and the reason.

Decision	Alternatives	Why this choice
Point at the button instead of describing it	Text instructions (version 2–3)	Users still couldn't find the button from text. Showing is clearer than telling.
One step at a time, fresh screenshot each step	Plan all steps up front	Screens change after every click (menus open, dialogs appear). A plan made on the first screenshot goes stale.
Let the AI choose with tools	Separate boxes for tasks and questions; always send a screenshot	One box is simpler for users. And a to-do never needs a screenshot, which is better for privacy.
Free AI models (Groq / Gemini)	Paid Claude API (earlier versions)	No credit card needed, so anyone (including students, the target users) can run it. Groq for speed, Gemini for precision.
Add a server	Keep everything on the desktop (version 2–3)	Needed for accounts, history across devices, scheduled emails, and keeping the AI key off the user's machine.
Use OCR to place the pointer	Trust the AI's own coordinates	Fast models are vague about exact positions. OCR is free, already in Windows, and pixel-exact for text.
SQLite database	PostgreSQL, MySQL	One file, nothing to install. Perfect for a small app; easy to swap later.
Electron for the desktop	Native Windows app; Tauri	Web skills (React) carry over; runs on Windows, Mac and Linux from one codebase.
Task list sent with every message	Give the AI a "list tasks" tool	The list is small, so "mark the dentist one done" works in a single round trip instead of two.
Demo mode with no AI key	Require a key to start	Anyone can try the whole flow offline, and automated tests run without a key.

What is a "tool", in AI terms?

Normally an AI just writes text. With **tool calling**, you give the AI a menu of actions it may request, such as *create_task* or *look_at_screen*. The AI replies "please run create_task with 'submit the form, 4pm'", your program actually does it, and tells the AI the result. The AI decides *what* to do; your code stays in control of *doing* it.

The seven concepts (capstone checklist)

The program required at least five of these. Alfred uses all seven. Be ready to explain any of them simply.

Concept	In plain English	How Alfred uses it
API endpoints	The “doors” the app knocks on to ask the server for something.	22 doors, e.g. log in, add a task, start a help session, download this week’s PDF.
Database	Organised long-term memory.	8 tables: users, tasks, help sessions, steps, messages, cache, reports, job runs.
Authentication	Proving who you are, and keeping each user’s data separate.	Sign-up and log-in. Passwords are stored scrambled (hashed), never as plain text. After log-in you get a digital pass (a JWT token), kept encrypted by the operating system’s keychain.
Background / scheduled jobs	Work that runs on a timer, not when someone clicks.	Weekly report every Monday 8am; email reminders the user missed (every 5 min); clean the cache (hourly); close abandoned sessions (every 30 min).
Reporting (PDF + email)	Producing a document and sending it.	A weekly PDF with totals, apps used, a step-by-step cheat sheet, and open to-dos, emailed to the user.
Caching	Remembering recent answers so you don’t do the same work twice.	Same question on the same screen = instant answer. The screen is identified by a “fingerprint” (perceptual hash), not by storing the image.
LLM integration	Using a large language model (an AI like ChatGPT) inside the app.	Tool-calling chat for tasks, and a vision model that reads screenshots. Includes a backup model if one fails, automatic retry when rate-limited, and an offline mock.

Say it in the interview

“A cache key is normally the exact input. Two screenshots of the same screen are never byte-identical, so I used a perceptual hash: a short fingerprint that stays the same when the image looks the same.”

8. Speed, privacy and quality

Making it fast (about one second)

The main idea: **never wait for two things one after the other if they can happen at the same time.**

- **Screenshot starts the moment you press Enter**, while Alfred is still deciding if it needs it.
- **A faster way to capture the screen on Windows**: about 0.1 seconds instead of 0.7 to 2 seconds with Electron's built-in method.
- **Obvious screen questions skip a step**: one AI call instead of two.
- **To-dos take one AI call**: the confirmation ("Reminder set for 4:00 PM") is written by code from the result, not by asking the AI again.
- **OCR runs while the AI is thinking**, so accurate pointing costs no extra time.
- **Fast models and warm connections**: Groq answers in under a second; the connection to each AI is kept open and reused.
- **Cache**: repeat questions on the same screen are instant.

Privacy by design

Privacy has been a core value since the pivot. These are firm rules, not settings:

- **No background screen capture.** Only when you send a message or press "next step". Never on a timer.
- **No screenshot leaves your computer unless the question is about your screen.**
- **No screenshots are ever saved**, on your computer or the server. The server keeps only a 64-character fingerprint for the cache.
- **The screen-facing part of the app never touches the internet directly.** It goes through Electron's secure main process, so your log-in token can't be stolen by the page.
- **No paid services and no tracking.**

Quality and testing

Kind of test	What it checks
67 desktop unit tests	Small pieces in isolation: tasks, reminders, date formatting, screen geometry, pointer snapping, routing.
35 server tests	Log-in, tasks, both AI tool loops, help sessions, cache, PDF, scheduled jobs. Run with a fake AI, so no key is needed.
End-to-end self-test	Drives the real app against a stand-in spreadsheet and saves screenshots, like a robot user.
Automatic checks (CI)	On every change pushed to GitHub, all tests, type checks and style checks run automatically.

9. Limits and what comes next

Being honest about limits is a strength in interviews. These were deliberately left out (the "scope guard" in the plan):

Not done yet	Why / what you'd do
Clicking for the user	Deliberately excluded: learning and safety. Would only ever be opt-in.
Voice input	Out of scope for now; a natural next step for hands-busy users.
Multiple monitors	Works on one display at a time; would need mapping between screens.
Mac and Linux real-mouse movement and fast capture	Windows first. The app works elsewhere, but the speed tricks and OCR are Windows features.
Offline AI model	Would improve privacy further, but local models are slower and less accurate today.
Classroom dashboard	Teachers could see what students struggle with. Needs careful privacy design.
Installer	Currently run from source; packaging and code signing are a separate release step.
Icon-only buttons	OCR reads text, so buttons with only an icon depend on the AI's own aim. Gemini is better at these.

Challenges worth mentioning

- **The AI pointed loosely.** Fast, free models gave rough positions. Solved by combining the AI's "what" (the button's name) with OCR's "where" (the exact pixels).
- **Slow screenshots.** Electron's built-in capture took up to 2 seconds. Switched to a lower-level Windows method (~0.1 s).
- **Alfred in its own screenshots.** The AI kept seeing Alfred's panel. Fixed by hiding Alfred's windows from the capture.
- **Free tiers have rate limits.** Handled with automatic waiting and retrying, plus a backup model.
- **Scope creep.** Version 1 did too much. The pivot was the most important decision in the project.

10. Likely interview questions

Short answers you can adapt. Speak them in your own words; don't recite.

Q: Tell me about this project.

Use the 30-second pitch on page 2, then the one-line tech summary.

Q: Why did you build it?

People get stuck in software they don't know. Tutorials say what to click but not where it is on your own screen. I wanted something that shows you, and teaches you, instead of doing it for you.

Q: What was the hardest part?

Getting the pointer exactly on the right button, fast. The free AI models were quick but imprecise about position. I let the AI say *which* button and used the OCR built into Windows to find *where* that text is. It runs while the AI is thinking, so it's accurate and free.

Q: What would you do differently?

Narrow the scope earlier. My first version was a big dashboard, and I deleted most of it. Now I write a one-page plan with a scope guard before building.

Q: Why one step at a time?

Every click changes the screen. A full plan made from the first screenshot goes stale as soon as a menu opens. A fresh look each step keeps it correct.

Q: How do you handle privacy?

Screenshots are only taken when the user asks, only sent if the question is about the screen, and never saved anywhere. The server keeps only a short fingerprint for caching.

Q: Why not let it click for the user?

Two reasons: the user learns, and the AI can never do damage by clicking the wrong thing.

Q: How did you make it fast?

Run everything that can overlap at the same time, skip AI calls that aren't needed, use faster models and a faster screenshot method, and cache repeat answers. About one second from Enter to pointer.

Q: How did you test it?

Around 100 automated tests across the app and server, an end-to-end robot test, and GitHub runs all checks on every change. The server tests use a fake AI so they don't need a key.

Q: Why did you switch from Claude to Gemini and Groq?

Cost and access. My users are students; free tiers with no credit card mean anyone can run it. I support both: Groq for speed, Gemini for precise pointing.

Q: How would it scale to many users?

Swap SQLite for PostgreSQL, move the cache to a shared store like Redis, run several copies of the server, and package a proper installer. The structure already separates these parts, so each swap is contained.

11. Glossary

Term	Meaning
API / endpoint	A set of “doors” a program offers so other programs can ask it for things. Each door is an endpoint.
Backend / server	The program running behind the scenes that stores data and does the heavy work.
Frontend / client	The part the user sees and touches. Here, the desktop app.
Electron	A toolkit for building desktop apps using website technology.
React	A popular library for building user interfaces out of reusable pieces.
TypeScript	JavaScript with types, which catch many mistakes before the program runs.
FastAPI	A Python framework for building web servers quickly.
SQLite	A small database stored in a single file.
LLM	Large language model: an AI that understands and writes text (ChatGPT, Gemini, Claude).
Vision model	An AI that can also look at images, such as a screenshot.
Tool calling	Letting an AI request actions from your program (e.g. “create a task”) instead of only writing text.
OCR	Optical character recognition: reading text out of an image.
Cache	A short-term memory of recent answers, to avoid repeating work.
Perceptual hash	A short fingerprint of an image that stays the same if the image looks the same.
Authentication	Checking who a user is (log-in).
Password hashing	Scrambling a password one-way so it can be checked but never read back.
JWT token	A signed digital pass the app shows on every request to prove you’re logged in.
Cron / scheduled job	A task that runs automatically at set times.
Rate limit	A cap on how many requests a free service allows per minute.
CI	Continuous integration: automatically running all tests whenever code changes.
Overlay	A transparent window on top of everything, used to draw the ghost cursor. Clicks pass straight through it.
Mock / demo mode	A fake AI that gives scripted answers, for trying the app or testing without a key.

One-page cheat sheet

The product

What: a desktop butler that points at the next thing to click, one step at a time, plus to-dos, reminders and a weekly PDF cheat sheet.

Who for: students and new office workers stuck in unfamiliar software.

Big idea: tutorials tell you *what*; Alfred shows you *where*.

The build

Desktop: Electron + React + TypeScript. Floating icon, panel, see-through overlay for the ghost cursor.

Server: Python FastAPI + SQLite. Accounts, tasks, history, cache, scheduled jobs, PDF reports, email.

AI: Groq (fast) or Gemini (precise), free tiers. Windows OCR snaps the pointer onto the exact text.

Seven decisions to remember

- Pointing beats describing.
- One step at a time on a fresh screenshot, because screens change.
- AI chooses with tools, so to-dos never need a screenshot.
- AI says *which* button, OCR says *where*.
- Free models so students can use it.
- Never clicks for you: learning and safety.
- Privacy: no background capture, no screenshots saved.

The story in one line

Dashboard (too broad) → pivot to one tiny icon → AI picks tools → guided pointer + server + free AI → about 1-second speed and OCR accuracy.

Numbers: 5 versions in 2 months · 22 API endpoints · 8 database tables · 7 of 7 capstone concepts · ~100 automated tests · about 1 s to point.